

Analyzing Malicious Windows Programs

Understanding API Interactions, Kernel Modes, and System Manipulation

Dr. Manu V.T.

February 19, 2026

The Windows API

- The Windows API is a broad set of functionality that governs the way that malware interacts with the Microsoft libraries.
- The Windows API is so extensive that developers of Windows-only applications have little need for third-party libraries.
- The Windows API uses certain terms, names, and conventions that you should become familiar with before turning to specific functions.

The Windows API & Notation

Hungarian Notation

- Windows API uses prefix naming to identify variable types.
- **DWORD (dw)**: 32-bit unsigned integer (e.g., dwSize).
- **WORD (w)**: 16-bit unsigned value.
- **LP (Long Pointer)**: Pointers to other types (e.g., LPCSTR is a pointer to a string).

Handles (H)

- References to objects (windows, processes, menus) opened in the OS.
- Not memory addresses; cannot be used for arithmetic.
- The only thing you can do with a handle is store it and use it in a later function call to refer to the same object.
- **HWND**: Handle to a Window.
- **HMODULE**: Handle to a Module.
- Used in functions like DestroyWindow(hwnd).

File System Manipulation:Standard I/O

Standard I/O

- **CreateFile:**

- This function is used to create and open files. It can open existing files, pipes, streams, and I/O devices, and create new files.
- The parameter **dwCreationDisposition** controls whether the CreateFile function creates a new file or opens an existing one.

- **ReadFile/WriteFile:**

- These functions are used for reading and writing to files. Both operate on files as a stream. When you first call ReadFile, you read the next several bytes from a file; the next time you call it, you read the next several bytes after that.
- For example, if you open a file and call ReadFile with a size of 40, the next time you call it, it will read beginning with the forty-first byte.
- As you can imagine, though, neither function makes it particularly easy to jump around within a file.

File System Manipulation: File Mapping

File Mapping

- File mappings are commonly used by malware writers because they allow a file to be loaded into memory and manipulated easily.
 - **CreateFileMapping** function loads a file from disk into memory.
 - **MapViewOfFile** function returns a pointer to the base address of the mapping, which can be used to access the file in memory.
 - The program calling these functions can use the pointer returned from **MapViewOfFile** to read and write anywhere in the file.
 - This feature is extremely handy when parsing a file format, because you can easily jump to different memory addresses.

Special Files

- Windows has a number of file types that can be accessed much like regular files, but that are not accessed by their drive letter and folder (like `c:\docs`).
- Malicious programs often use special files.
- Some special files can be stealthier than regular ones because they don't show up in directory listings.
- Certain special files can provide greater access to system hardware and internal data.
- Special files can be passed as strings to any of the file-manipulation functions, and will operate on a file as if it were a normal file.

Special Files

Shared Files

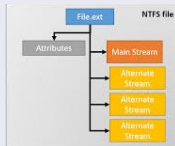
- Shared files are special files with names that start with `\\serverName\share` or `\\?\serverName\share`.
- They access directories or files in a shared folder stored on a network.
- The `\\?\` prefix tells the OS to disable all string parsing, and it allows access to longer filenames.

Files Accessible via Namespaces

- Additional files are accessible via namespaces within the OS. Namespaces can be thought of as a fixed number of folders, each storing different types of objects
 - The Win32 device namespace, with the prefix `\\.\`
 - A program can use the `\\.\PhysicalDisk1` to directly access `PhysicalDisk1` while ignoring its file system, allowing it to modify the disk in ways that are not possible through the normal API.
 - Using this method, the malware might be able to read and write data to an unallocated sector without creating or accessing files, which allows it to avoid detection by antivirus and security programs.

ADS

- The Alternate Data Streams (ADS) feature allows additional data to be added to an existing file within NTFS, essentially adding one file to another.
- The extra data does not show up in a directory listing, and it is not shown when displaying the contents of the file; it's visible only when you access the stream.
- ADS data is named according to the convention `normalFile.txt:Stream::$DATA`, which allows a program to read and write to a stream.
- Malware authors like ADS because it can be used to hide data.

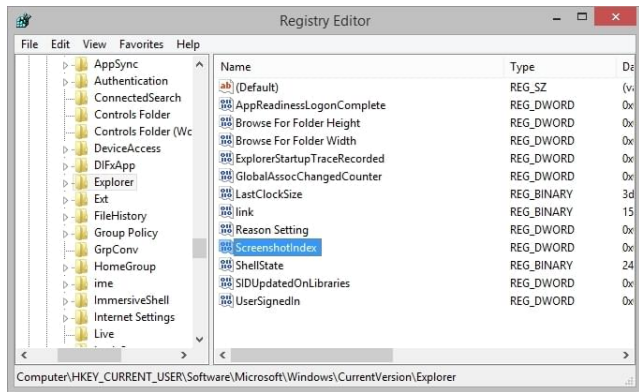


The Windows Registry

- The Windows registry is used to store OS and program configuration information, such as settings and options.
- Like the file system, it is a good source of host-based indicators and can reveal useful information about the malware's functionality.
- Early versions of Windows used .ini files to store configuration information.
- The registry was created as a hierarchical database of information to improve performance, and its importance has grown as more applications use it to store information.
- Nearly all Windows configuration information is stored in the registry, including networking, driver, startup, user account, and other information

The Windows Registry

- Malware often uses the registry for persistence or configuration data.
- The malware adds entries into the registry that will allow it to run automatically when the computer boots.
- The registry is so large that there are many ways for malware to use it for persistence.



The Windows Registry Terms

- **Root key.** The registry is divided into five top-level sections called root keys. Sometimes, the terms HKEY and hive are also used. Each of the root keys has a particular purpose, as explained next.
- **Subkey.** A subkey is like a subfolder within a folder.
- **Key.** A key is a folder in the registry that can contain additional folders or values. The root keys and subkeys are both keys.
- **Value entry.** A value entry is an ordered pair with a name and value.
- **Value or data.** The value or data is the data stored in a registry entry.

Registry Root Keys

- **HKEY_LOCAL_MACHINE (HKLM)** — Stores settings that are global to the local machine.
- **HKEY_CURRENT_USER (HKCU)** — Stores settings specific to the current user.
- **HKEY_CLASSES_ROOT** — Stores information defining file types and COM class registrations.
- **HKEY_CURRENT_CONFIG** — Stores settings about the current hardware configuration, specifically differences between the current and the standard configuration.
- **HKEY_USERS** — Defines settings for the default user, new users, and all loaded user profiles.
- The two most commonly used root keys are **HKEY_LOCAL_MACHINE (HKLM)** and **HKEY_CURRENT_USER (HKCU)**
- They are commonly referred to by their abbreviations, **HKLM** and **HKCU**.

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Run

- Contains a series of values corresponding to executables that are started automatically when a user logs in.
- The root key is **HKEY_LOCAL_MACHINE**, which stores the subkeys:
 - SOFTWARE
 - Microsoft
 - Windows
 - CurrentVersion
 - Run

The Windows Registry

- Malware often uses registry functions that are part of the Windows API in order to modify the registry to run automatically when the system boots.
- The following are the most common registry functions:
 - **RegOpenKeyEx**. Opens a registry for editing and querying. There are functions that allow you to query and edit a registry key without opening it first, but most programs use RegOpenKeyEx anyway.
 - **RegSetValueEx**. Adds a new value to the registry and sets its data.
 - **RegGetValue**. Returns the data for a value entry in the registry.

Analyzing Registry Code in Practice

- Real malware code opening the Run key from the registry and adding a value so that the program runs each time Windows starts.
- The **RegSetValueEx** function, which takes six parameters, edits a registry value entry or creates a new one if it does not exist.

```

0040286F  push    2                ; samDesired
00402871  push    eax              ; ulOptions
00402872  push    offset SubKey    ; "Software\\Microsoft\\Windows..."
00402877  push    HKEY_LOCAL_MACHINE ; hKey
0040287C  call    esi              ; RegOpenKeyExW
0040287E  test    eax, eax
00402880  jnz     short loc_4028C5
...
00402886  push    ecx              ; lpString
00402887  mov     bl, 1
00402889  call    ds:strlenW
0040288F  lea     edx, [eax+eax+2]
00402893  push    edx              ; cbData
00402894  mov     edx, [esp+428h+hKey]
00402898  lea     eax, [esp+428h+Data]
0040289C  push    eax              ; lpData
0040289D  push    1                ; dwType
0040289F  push    0                ; Reserved
004028A1  lea     ecx, [esp+434h+ValueName]
004028A8  push    ecx              ; lpValueName
004028A9  push    edx              ; hKey
004028AA  call    ds:RegSetValueExW

```

- **The Port (The “Mailbox”)**

- A port is a numerical identifier (ranging from 0 to 65535) used to distinguish between different services or applications running on a single host.
- **Purpose:** It allows the Operating System to direct incoming data to the correct application (e.g., Port 80 for HTTP, Port 443 for HTTPS).

- **The Socket (The “Interface”)**

- A socket is a software structure or endpoint created by a program to send and receive data.
- **Definition:** It is the combination of an IP Address + Port Number.
- **Networking Roles:** There are two sides to a networking program using sockets: the server side, which maintains an open socket waiting for connections, and the client side, which connects to a waiting socket.
- **Implementation:** In Windows, these are primarily implemented in the Winsock libraries, primarily in `ws2_32.dll`.

Feature	Port	Socket
What it is	A 16-bit numerical identifier.	A software endpoint for communication.
Represented by	A single number (e.g., 443).	IP Address + Port (e.g., 192.168.1.1:443).
Function	Identifies a specific service on a host.	Facilitates the actual data transfer between apps.
Malware Context	Malware may hide by using unusual ports.	Analysts look for specific call sequences like <code>socket</code> → <code>bind</code> → <code>listen</code> .

Networking APIs: Berkeley Compatible Sockets

- Berkeley compatible sockets' network functionality in Windows is implemented in the Winsock libraries, primarily in `ws2_32.dll`.
- **Server:** `socket`, `bind`, `listen`, `accept`.
- **Client:** `socket`, `connect`, `send`, `recv`.
- Requires `WSAStartup` initialization.

Function	Description
<code>socket</code>	Creates a socket
<code>bind</code>	Attaches a socket to a particular port, prior to the <code>accept</code> call
<code>listen</code>	Indicates that a socket will be listening for incoming connections
<code>accept</code>	Opens a connection to a remote socket and accepts the connection
<code>connect</code>	Opens a connection to a remote socket; the remote socket must be waiting for the connection
<code>recv</code>	Receives data from the remote socket
<code>send</code>	Sends data to the remote socket

Berkeley Compatible Sockets

- There are always two sides to a networking program: the server side, which maintains an open socket waiting for incoming connections, and the client side, which connects to a waiting socket.
- Malware can be either one of these.
- In the case of client-side applications that connect to a remote socket, you will see the `socket` call followed by the `connect` call, followed by `send` and `recv` as necessary.
- For a service application that listens for incoming connections, the `socket`, `bind`, `listen`, and `accept` functions are called in that order, followed by `send` and `recv`, as necessary.
- This pattern is common to both malicious and nonmalicious programs.

```

00401041    push    ecx                      ; lpWSAData
00401042    push    202h                    ; wVersionRequested
00401047    mov     word ptr [esp+250h+name.sa_data], ax
0040104C    call    ds:WSAStartup
00401052    push    0                        ; protocol
00401054    push    1                        ; type
00401056    push    2                        ; af
00401058    call    ds:socket
0040105E    push    10h                     ; namelen
00401060    lea     edx, [esp+24h+name]
00401064    mov     ebx, eax
00401066    push    edx                      ; name
00401067    push    ebx                      ; s
00401068    call    ds:bind
0040106E    mov     esi, ds:listen
00401074    push    5                        ; backlog
00401076    push    ebx                      ; s
00401077    call    esi                      ; listen
00401079    lea     eax, [esp+248h+addrlen]
0040107D    push    eax                      ; addrlen
0040107E    lea     ecx, [esp+24Ch+hostshort]
00401082    push    ecx                      ; addr
00401083    push    ebx                      ; s
00401084    call    ds:accept

```

- First, `WSAStartup` initializes the Win32 sockets system.
- The `ds:` prefix indicates that the function address is being retrieved from the Data Segment, specifically the Import Address Table (IAT), allowing the program to dynamically link to functions within `ws2_32.dll` at runtime.

- First, `WSAStartup` initializes the Win32 sockets system.
- The `ds:` prefix indicates that the function address is being retrieved from the Data Segment, specifically the Import Address Table (IAT), allowing the program to dynamically link to functions within `ws2_32.dll` at runtime.
- Then a socket is created with `socket`.
- The `bind` function attaches the socket to a port.
- The `listen` call sets up the socket to listen.
- The `accept` call hangs, waiting for a connection from a remote socket.

Networking APIs: WinINet API: (*Wininet.dll*)

- Higher-level application layer protocols (HTTP, FTP).
- **InternetOpen:** Initializes connection.
- **InternetOpenUrl:** Connects to specific URL.
- **InternetReadFile:** Downloads data like a local file.

Dynamic Link Libraries (DLLs)

DLLs allow code sharing but are heavily exploited by malware.

Storing Code Hiding malicious logic inside a DLL loaded by legitimate processes.

Dependency Importing functions from standard DLLs (kernel32.dll).

Injection Using LoadLibrary and GetProcAddress to dynamically resolve and execute functions.

• How Malware Authors Use DLLs

- **To store malicious code:** Malware authors may store malicious code in a DLL rather than an .exe because while a process can only contain one .exe, it can load multiple DLLs. This allows malware to load itself into other processes.
- **By using Windows DLLs:** Nearly all malware interacts with the OS using standard Windows DLLs (like ws2_32.dll or kernel32.dll). The specific functions a program imports from these libraries provide significant insight into its intended behavior.
- **By using third-party DLLs:** Malware may import functions from third-party programs (such as Mozilla Firefox DLLs) to interact with those applications or use custom DLLs to include functionality not present on the victim's system, such as specific encryption libraries.

Basic DLL Structure

- DLL files closely resemble .exe files and use the PE (Portable Executable) format.
- A single flag in the PE header distinguishes a DLL from an executable.
- DLLs typically have more exports and fewer imports than executables.
- The main entry function is `DllMain`, specified in the PE header as the entry point.
- `DllMain` is invoked when:
 - A process loads or unloads the DLL
 - A thread is created or exits
- These notifications allow the DLL to manage per-process and per-thread resources.
- Most DLLs ignore thread-related calls unless thread-specific resources are required.
- Presence of per-thread resource handling can hint at the DLL's purpose during analysis.

Processes: Definition

- A process is a program currently being executed by Windows.
- Each process contains:
 - One or more threads executed by the CPU
 - Its own memory space
 - Its own open handles and resources

Malware and Process Usage

- Malware can execute code by:
 - Creating new processes
 - Modifying existing processes
- Older malware used independent processes.
- Modern malware commonly injects code into legitimate processes.

Purpose of Processes

- Processes act as containers for:
 - CPU
 - Memory
 - File system access
 - Hardware resources
- They prevent programs from interfering with each other.
- Improve system stability by isolating crashes and errors.

Process Memory Spaces

- Each process receives its own virtual memory space.
- Memory addresses are unique per process.
- The OS maps virtual memory to physical memory.

Memory Address Context

- Different processes may use the same virtual address.
- Example:
 - Process A stores data at 0x00400000
 - Process B may also use 0x00400000
- Physical memory locations differ.
- Addresses are meaningful only within a process.

Security Implications

- Malware accessing memory affects only its process space.
- Other processes remain unaffected.
- Memory isolation limits unintended cross-process impact.

Creating a New Process

- Malware commonly creates new processes using the `CreateProcess` function.
- `CreateProcess` provides extensive control over:
 - How the process is initialized
 - Which program is executed
 - Process attributes and environment
- Common malicious uses:
 - Executing malicious code in a separate process
 - Bypassing host-based firewalls and security controls
 - Launching legitimate programs (e.g., browsers) for malicious activity
- Malware can create remote shells using a single `CreateProcess` call.
- The `STARTUPINFO` structure allows control of:
 - Standard input
 - Standard output
 - Standard error streams
- Redirecting these streams to a socket enables remote command execution.

CreateProcess Remote Shell Example

- Demonstrates using CreateProcess to spawn a simple remote shell.
- A socket is opened to a remote host prior to this code.
- The socket handle is stored and inserted into the STARTUPINFO structure.
- Standard input, output, and error streams are redirected to the socket.
- All process I/O is routed through the remote connection.

```
004010DA  mov     eax, dword ptr [esp+58h+SocketHandle]
004010DE  lea     edx, [esp+58h+StartupInfo]
004010E2  push    ecx                ; lpProcessInformation
004010E3  push    edx                ; lpStartupInfo
004010E4  1mov    [esp+60h+StartupInfo.hStdError], eax
004010E8  2mov    [esp+60h+StartupInfo.hStdOutput], eax
004010EC  3mov    [esp+60h+StartupInfo.hStdInput], eax
004010F0  4mov    eax, dword_403098
004010F5  push    0                  ; lpCurrentDirectory
004010F7  push    0                  ; lpEnvironment
004010F9  push    0                  ; dwCreationFlags
004010FB  mov     dword ptr [esp+6Ch+CommandLine], eax
004010FF  push    1                  ; bInheritHandles
00401101  push    0                  ; lpThreadAttributes
00401103  lea     eax, [esp+74h+CommandLine]
00401107  push    0                  ; lpProcessAttributes
00401109  5push    eax                ; lpCommandLine
0040110A  push    0                  ; lpApplicationName
0040110C  mov     [esp+80h+StartupInfo.dwFlags], 101h
00401114  6call    ds:CreateProcessA
```

Analyzing CreateProcess Redirection

- The `CreateProcess` call spawns a new process with:
 - Standard input redirected to a socket
 - Standard output redirected to a socket
- This enables remote command execution.
- To identify the remote host:
 - Locate where the socket is initialized in the code
- To determine which program is executed:
 - Navigate to the string at the referenced memory address
 - Inspect it in a disassembler (e.g., IDA Pro)

Threads

- Processes are the container for execution, but threads are what the Windows OS executes.
- Threads are independent sequences of instructions that are executed by the CPU without waiting for other threads.
- A process contains one or more threads, which execute part of the code within a process.
- Threads within a process all share the same memory space, but each has its own processor registers and stack.

Thread Context

- When one thread is running, it has complete control of the CPU, or the CPU core, and other threads cannot affect the state of the CPU or core.
- When a thread changes the value of a register in a CPU, it does not affect any other threads.
- Before an OS switches between threads, all values in the CPU are saved in a structure called the thread context.
- The OS then loads the thread context of a new thread into the CPU and executes the new thread.

Accessing a local variable and pushing it on the stack

```
004010DE  lea 1edx, [esp+58h]  
004010E2  push    edx
```

- Code at 1 accesses a local variable ($\text{esp}+58\text{h}$) and stores it in EDX, and then pushes EDX onto the stack.
- Now, if another thread were to run some code in between these two instructions, and that code modified EDX, the value of EDX would be wrong, and the code would not execute properly.
- When thread-context switching is used, if another thread runs in between these two instructions, the value of EDX is stored in the thread context.
- When the thread starts again and executes the push instruction, the thread context is restored, and EDX stores the proper value again.
- In this way, no thread can interfere with the registers or flags from another thread.

Creating a Thread

- The `CreateThread` function is used to create new threads.
- The function's caller specifies a start address, which is often called the start function.
- Execution begins at the start address and continues until the function returns, although the function does not need to return, and the thread can run until the process ends.
- When analyzing code that calls `CreateThread`, you will need to analyze the start function in addition to analyzing the rest of the code in the function that calls `CreateThread`.
- The caller of `CreateThread` can specify the function where the thread starts and a single parameter to be passed to the start function.
- The parameter can be any value, depending on the function where the thread will start.

CreateThread Usage in Malware

Malware frequently utilizes the `CreateThread` API to execute concurrent malicious activities. Two common methods include:

① Loading Malicious Libraries

- `CreateThread` is called with the address of `LoadLibrary` specified as the start address.
- The argument passed is the name of the malicious DLL to be loaded.
- The DLL is loaded into the process memory, and `DllMain` is executed.

② Input/Output Redirection

- Malware creates two separate threads for I/O:
 - ① One thread listens on a **socket or pipe** and redirects data to the process's **standard input**.
 - ② The other thread reads from **standard output** and sends it back to the **socket or pipe**.
- **Goal:** Seamless communication between the attacker and the running application.

```

004016EE lea     eax, [ebp+ThreadId]
004016F4 push    eax             ; lpThreadId
004016F5 push    0               ; dwCreationFlags
004016F7 push    0               ; lpParameter
004016F9 push    1offset ThreadFunction1 ; lpStartAddress
004016FE push    0               ; dwStackSize
00401700 lea     ecx, [ebp+ThreadAttributes]
00401706 push    ecx             ; lpThreadAttributes
00401707 call    2ds:CreateThread
0040170D mov     [ebp+var_59C], eax
00401713 lea     edx, [ebp+ThreadId]
00401719 push    edx             ; lpThreadId
0040171A push    0               ; dwCreationFlags
0040171C push    0               ; lpParameter
0040171E push    3offset ThreadFunction2 ; lpStartAddress
00401723 push    0               ; dwStackSize
00401725 lea     eax, [ebp+ThreadAttributes]
0040172B push    eax             ; lpThreadAttributes
0040172C call    4ds:CreateThread

```

```

...
004012C5 call    ds:ReadFile
...
00401356 call    ds:send
...

```

```

...
004011F2 call    ds:recv
...
00401271 call    ds:WriteFile
...

```

Processes & Threads

CreateProcess

Creates a new container. Malware uses STARTUPINFO to redirect std I/O to sockets (Remote Shells).

CreateThread

Executes independent instruction sequences. Used to handle Input/Output separately for C2 communication.

Context Switching

OS saves/restores CPU register states (Context) to ensure thread isolation.

1

Single Instance

Mutex Objects (Mutual Exclusions)

- Coordinate access to shared resources.
- Malware uses **hard-coded named mutexes** to ensure only one instance of the infection runs.
- *Indicator:* Calls to `CreateMutex` followed by `OpenMutex`.

Windows Services

- **Background Execution:** Run without user intervention. Ideal for persistence.
- **Privilege Level:** Often run as **SYSTEM** (higher than Admin).
- **svchost.exe:**
 - WIN32_SHARE_PROCESS service type.
 - Multiple services run inside a single `svchost.exe` to mask activity.

Component Object Model (COM)

COM Architecture

- Standard allowing software components to interact.
- **GUIDs:** Uses CLSIDs and IIDs to identify objects.
- **CoCreateInstance:** API to instantiate an object.

Malicious Use

- Malware acts as COM client to automate IE.
- **BHOs (Browser Helper Objects):** Run inside the browser process.
- **Stealth:** Code runs within trusted `iexplore.exe`.

Kernel vs. User Mode

User Mode (Ring 3)

- Restricted access.
- Processes isolated; crashes are contained.

Kernel Mode (Ring 0)

- Unrestricted access to hardware/memory.
- **Rootkits:** Hide from user-mode AV.
- Bugs cause BSOD.

Bypassing Windows API

Bypassing `kernel32.dll` to call `ntdll.dll` directly allows malware to perform actions not exposed by standard APIs and evade security monitors.

Critical Functions:

`NtQuerySystemInformation`, `NtTerminateProcess`